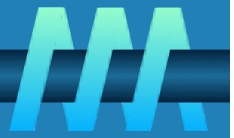
Análisis y Diseño de Algoritmos I

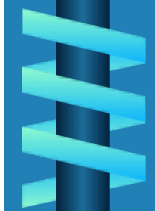
Transform-and-Conquer

(notas basadas en el material adicional de “The Design & Analysis of Algoritmos” (Levitin 2003).

Transform-and-Conquer

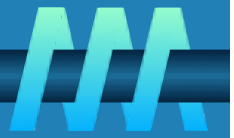


That's the secret to life . . . replace one worry with another.



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Transform-and-Conquer



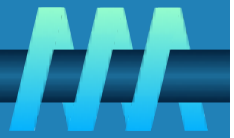
That's the secret to life . . . replace one worry with another.

Primero, en la etapa de transformación, la instancia del problema se modifica para que, por una u otra razón, sea más fácil de resolver. Luego, en la segunda etapa o de resolución, se procede a resolverlo.



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Transform-and-Conquer



That's the secret to life . . . replace one worry with another.

Primero, en la etapa de transformación, la instancia del problema se modifica para que, por una u otra razón, sea más fácil de resolver. Luego, en la segunda etapa o de resolución, se procede a resolverlo.

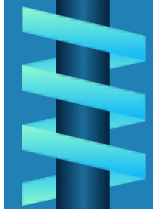


Copyright © 2007 Pearson Addison-Wesley. All rights reserved.



Es un grupo de técnicas que resuelven un problema mediante transformación:

- **A un problema más simple/más conveniente instancia del mismo problema (*instance simplification*)**
- **A una representación diferente de la misma instancia (*representation change*)**
- **A un problema diferente para el cual ya existe un algoritmo que lo resuelve (*problem reduction*)**



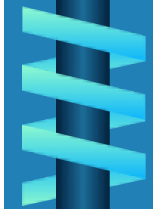
Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Transform and Conquer

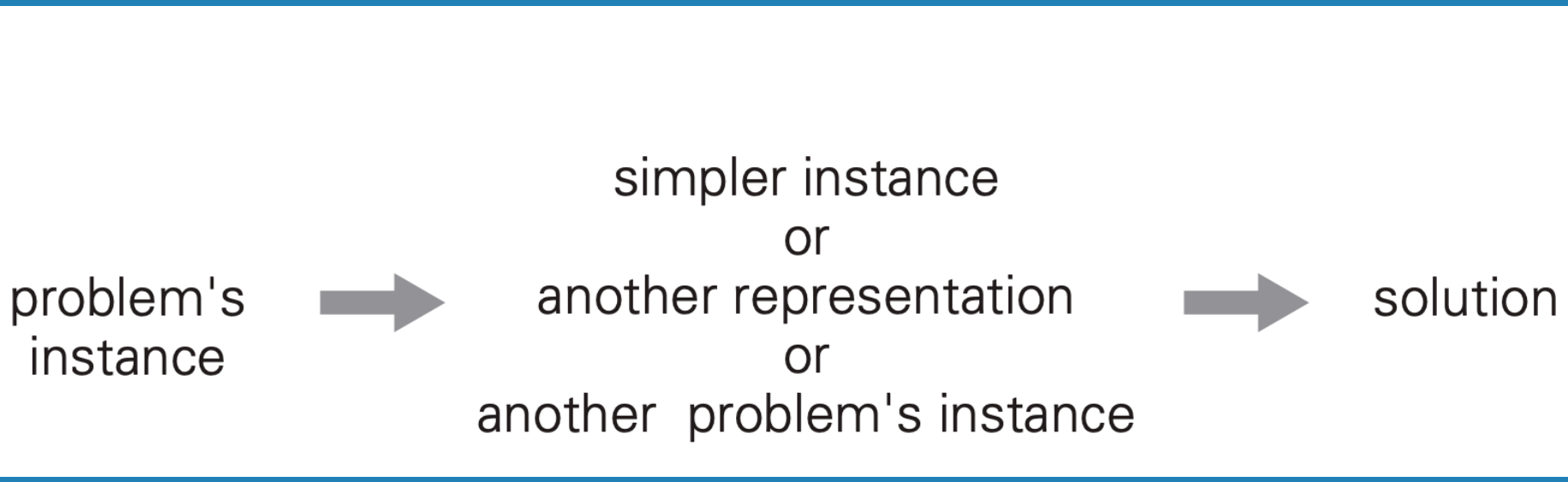


Es un grupo de técnicas que resuelven un problema mediante transformación:

- **A un problema más simple/más conveniente instancia del mismo problema (*instance simplification*)**
- **A una representación diferente de la misma instancia (*representation change*)**
- **A un problema diferente para el cual ya existe un algoritmo que lo resuelve (*problem reduction*)**

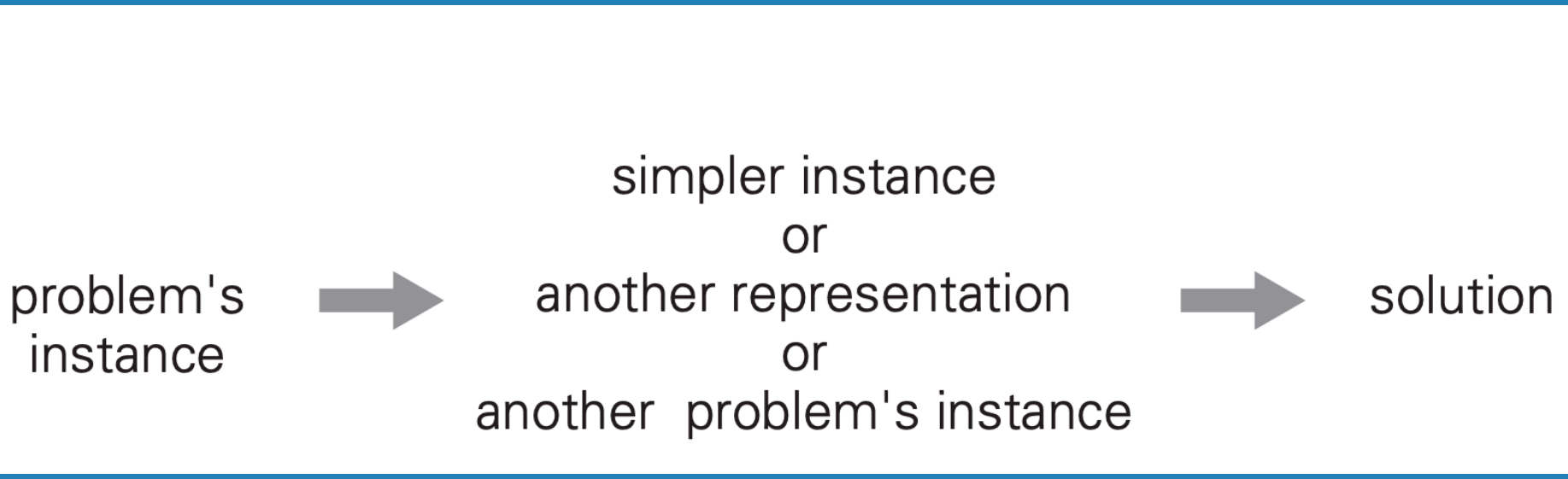


Copyright © 2007 Pearson Addison-Wesley. All rights reserved.



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Transform and Conquer



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Presorting



Resolver una instancia de un problema transformándolo a un problema más simple/más conveniente instancia del mismo problema (*instance simplification*)

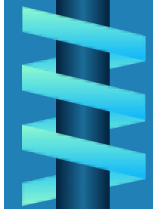
Presorting

Muchos problemas que involucran listas son más fácil cuando la lista esta ordenada

- Búsqueda
- Chequear que todos los elementos de una lista son distintos.
- Cálculo de la moda

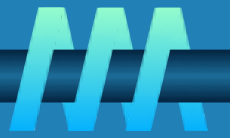
También:

- Orden topológico ayuda resolver algunos problemas sobre grafos
- Presorting también es usado en varios problemas geométricos: ejemplo Closest pair problem.



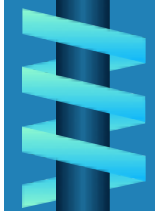
Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Qué tan eficiente podemos ordenar?



La eficiencia de los algoritmos que implican ordenamiento depende de la eficiencia del propio proceso de ordenamiento.

Se requieren aproximadamente n $\log_2$ n comparaciones en el peor de los casos para ordenar una lista de tamaño *n* mediante cualquier algoritmo general basado en comparaciones.



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Búsqueda con presorting



Problema: Buscar un valor dado K en A[0..n−1]

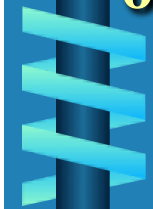
Algoritmo basado en presorting:

- **Etapa 1: Ordenar el arreglo utilizando un algoritmo de ordenamiento eficiente.**
- **Etapa 2:** Aplicar **búsqueda binaria**.

Eficiencia: Θ(n log n) + O(log n) = Θ(n log n)

¿Bueno o malo?

¿Por qué tenemos nuestros diccionarios, guías telefónicas, etc., ordenados?



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Element Uniqueness con presorting



Algoritmo basado en preordenamiento (Presorting-based algorithm):

- **Etapa 1: Ordenar el arreglo utilizando un algoritmo de ordenamiento eficiente (por ejemplo, *mergesort*).**
- **Etapa 2: Recorrer el arreglo para verificar los pares de elementos adyacentes.**

Eficiencia: $\Theta(n \log n) + O(n) = \Theta(n \log n)$

Algoritmo de fuerza bruta (Brute force algorithm):

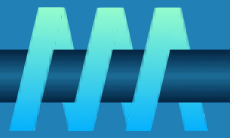
- Comparar **todos los pares de elementos**.
- **Eficiencia:** $O(n^2)$



¿Otro algoritmo posible? Hashing

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Ejercicio



Considere el problema de calcular la moda de una secuencia. La moda es el valor que aparece con mayor frecuencia en una lista de números. Por ejemplo, para la lista 5, 1, 5, 7, 6, 5, 7, la moda es 5.
(Si varios valores diferentes aparecen con la misma frecuencia máxima, cualquiera de ellos puede considerarse una moda.)



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Ejercicio



Considere el problema de calcular la moda de una secuencia. La moda es el valor que aparece con mayor frecuencia en una lista de números. Por ejemplo, para la lista 5, 1, 5, 7, 6, 5, 7, la moda es 5.
(Si varios valores diferentes aparecen con la misma frecuencia máxima, cualquiera de ellos puede considerarse una moda.)

¿Solución Fuerza Bruta?



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Ejercicio



Considere el problema de calcular la moda de una secuencia. La moda es el valor que aparece con mayor frecuencia en una lista de números. Por ejemplo, para la lista 5, 1, 5, 7, 6, 5, 7, la moda es 5.
(Si varios valores diferentes aparecen con la misma frecuencia máxima, cualquiera de ellos puede considerarse una moda.)

¿Solución Fuerza Bruta?

¿Tiempo de Ejecución?



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Ejercicio

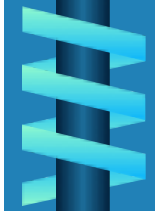


Moda

Algoritmo: CalcularModaFuerzaBruta(Secuencia)

Entrada: Una secuencia A de n números.
Salida: El valor que aparece con mayor frecuencia (la moda).

1. Inicializar max_frecuencia <- 0
2. Inicializar moda <- Nulo
3. **Para** cada elemento i desde 0 hasta n - 1:
    Inicializar frecuencia_actual <- 0
    Para cada elemento j desde 0 hasta n - 1:
        Si A[i] == A[j]:
            Incrementar frecuencia_actual en 1
    Si frecuencia_actual > max_frecuencia:
        max_frecuencia <-frecuencia_actual
        moda <- A[i]
4. Devolver moda



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Ejercicio



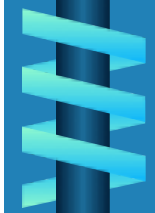
Moda

Solución Fuerza Bruta

Algoritmo: CalcularModaFuerzaBruta(Secuencia)

Entrada: Una secuencia A de n números.
Salida: El valor que aparece con mayor frecuencia (la moda).

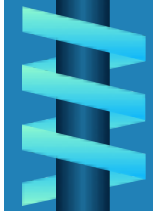
1. Inicializar max_frecuencia <- 0
2. Inicializar moda <- Nulo
3. **Para** cada elemento i desde 0 hasta n - 1:
    Inicializar frecuencia_actual <- 0
    Para cada elemento j desde 0 hasta n - 1:
        Si A[i] == A[j]:
            Incrementar frecuencia_actual en 1
    Si frecuencia_actual > max_frecuencia:
        max_frecuencia <-frecuencia_actual
        moda <- A[i]
4. Devolver moda



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - moda





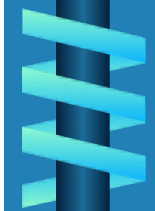
Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - moda



ALGORITHM *PresortMode*$(A[0..n-1])$

//Computes the mode of an array by sorting it first
//Input: An array $A[0..n-1]$ of orderable elements
//Output: The array's mode
sort the array A
$i \leftarrow 0$ //current run begins at position i
$modefrequency \leftarrow 0$ //highest frequency seen so far
while $i \leq n-1$ **do**
    $runlength \leftarrow 1$; $runvalue \leftarrow A[i]$
    while $i + runlength \leq n-1$ **and** $A[i + runlength] = runvalue$
        $runlength \leftarrow runlength + 1$
    if $runlength > modefrequency$
        $modefrequency \leftarrow runlength$; $modevalue \leftarrow runvalue$
    $i \leftarrow i + runlength$
return $modevalue$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

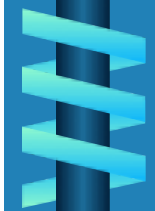
Instance simplification - moda



ALGORITHM *PresortMode*$(A[0..n-1])$

//Computes the mode of an array by sorting it first
//Input: An array $A[0..n-1]$ of orderable elements
//Output: The array's mode
sort the array A
$i \leftarrow 0$ //current run begins at position i
$modefrequency \leftarrow 0$ //highest frequency seen so far
while $i \leq n-1$ **do**
    $runlength \leftarrow 1$; $runvalue \leftarrow A[i]$
    while $i + runlength \leq n-1$ **and** $A[i + runlength] = runvalue$
        $runlength \leftarrow runlength + 1$
    if $runlength > modefrequency$
        $modefrequency \leftarrow runlength$; $modevalue \leftarrow runvalue$
    $i \leftarrow i + runlength$
return $modevalue$

¿Tiempo de Ejecución?



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - moda



ALGORITHM *PresortMode*$(A[0..n-1])$

```
//Computes the mode of an array by sorting it first
//Input: An array A[0..n − 1] of orderable elements
//Output: The array's mode
sort the array A
i ← 0                                  //current run begins at position i
modefrequency ← 0                      //highest frequency seen so far
while i ≤ n − 1 do
    runlength ← 1;   runvalue ← A[i]
    while i + runlength ≤ n − 1 and A[i + runlength] = runvalue
        runlength ← runlength + 1
    if runlength > modefrequency
        modefrequency ← runlength;   modevalue ← runvalue
    i ← i + runlength
return modevalue
```

¿Tiempo de Ejecución?

¿Alguna solución más eficiente?

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Ejercicio



Considere el siguiente problema. Dados dos conjuntos S_1 y S_2 (cada uno de tamaño n) y un número x, se quiere saber si existe un par de elementos, uno de S_1 y el otro de S_2, tales que su suma es x.



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Ejercicio



Considere el siguiente problema. Dados dos conjuntos S_1 y S_2 (cada uno de tamaño n) y un número x, se quiere saber si existe un par de elementos, uno de S_1 y el otro de S_2, tales que su suma es x.

```
TwoSum(S1, S2, x):
    L = ordenar(S2)
     para cada a en S1:
         b = x - a
         si busquedaBinaria(L, b) == true:
             retornar true
     retornar false
```



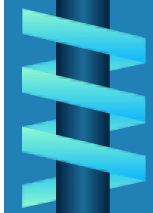
Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification - Ejercicio



Considere el siguiente problema. Dados dos conjuntos S_1 y S_2 (cada uno de tamaño n) y un número x, se quiere saber si existe un par de elementos, uno de S_1 y el otro de S_2, tales que su suma es x.

```
TwoSum(S1, S2, x):
    L = ordenar(S2)
     para cada a en S1:
         b = x - a
         si busquedaBinaria(L, b) == true:
             retornar true
     retornar false
```



Tiempo de Ejecución?

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.



Dado: un sistema de **n ecuaciones lineales** con **n incógnitas** y una **matriz de coeficientes arbitraria**.

Transformar en: un **sistema equivalente** de **n ecuaciones lineales** con **n incógnitas**, cuya **matriz de coeficientes sea triangular superior**.

Resolver este último sistema mediante sustituciones, comenzando con la última ecuación y avanzando hacia la primera.

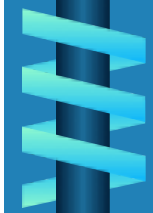
$$a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1$$
$$a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n = b_2$$

$$a_{1,1}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1$$
$$a_{22}x_2 + \ldots + a_{2n}x_n = b_2$$



$$a_{n1}x_1 + a_{n2}x_2 + \ldots + a_{nn}x_n = b_n$$

$$a_{nn}x_n = b_n$$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance simplification – Eliminación Gauseana



Dado: un sistema de **n ecuaciones lineales** con **n incógnitas** y una **matriz de coeficientes arbitraria**.

Transformar en: un **sistema equivalente** de **n ecuaciones lineales** con **n incógnitas**, cuya **matriz de coeficientes sea triangular superior**.

Resolver este último sistema mediante sustituciones, comenzando con la última ecuación y avanzando hacia la primera.

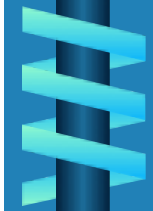
$$a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1$$
$$a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n = b_2$$

$$a_{1,1}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1$$
$$a_{22}x_2 + \ldots + a_{2n}x_n = b_2$$



$$a_{n1}x_1 + a_{n2}x_2 + \ldots + a_{nn}x_n = b_n$$

$$a_{nn}x_n = b_n$$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Ejemplo

$$\begin{aligned} 2x_1 - x_2 + x_3 &= 1 \\ 4x_1 + x_2 - x_3 &= 5 \\ x_1 + x_2 + x_3 &= 0. \end{aligned}$$



$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 4 & 1 & -1 & 5 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Eliminar x1 de la fila 2: Calculamos el factor $\frac{4}{2} = 2$ y hacemos

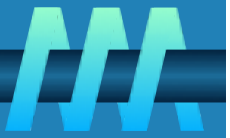
$R_2 \leftarrow R_2 - 2R_1$

$$2R_1 = [4, -2, 2 \mid 2],$$

$$R_2 - 2R_1 = [4-4, 1-(-2), -1-2 \mid 5-2] = [0, 3, -3 \mid 3].$$

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Gaussian Elimination (cont.)



Ejemplo

$$\begin{aligned} 2x_1 - x_2 + x_3 &= 1 \\ 4x_1 + x_2 - x_3 &= 5 \\ x_1 + x_2 + x_3 &= 0. \end{aligned}$$

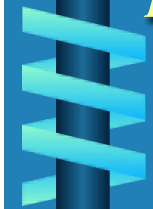


$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 4 & 1 & -1 & 5 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Eliminar x1 de la fila 2: Calculamos el factor $\frac{4}{2} = 2$ y hacemos

$R_2 \leftarrow R_2 - 2R_1$

$$2R_1 = [4, -2, 2 \mid 2],$$
$$R_2 - 2R_1 = [4-4, 1-(-2), -1-2 \mid 5-2] = [0, 3, -3 \mid 3].$$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Ejemplo

$$\begin{aligned} 2x_1 - x_2 + x_3 &= 1 \\ 4x_1 + x_2 - x_3 &= 5 \\ x_1 + x_2 + x_3 &= 0. \end{aligned}$$



$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 4 & 1 & -1 & 5 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Eliminar x1 de la fila 3: Calculamos el factor $\frac{1}{2}$ y hacemos

$$R_3 \leftarrow R_3 - \frac{1}{2}R_1$$

$$\frac{1}{2}R_1 = [\,1,\ -\tfrac{1}{2}, \tfrac{1}{2} \mid \tfrac{1}{2}\,],$$

$$R_3 - \tfrac{1}{2}R_1 = [\,1-1,\ 1-(-\tfrac{1}{2}),\ 1-\tfrac{1}{2} \mid 0-\tfrac{1}{2}\,] = [\,0, \tfrac{3}{2}, \tfrac{1}{2} \mid\ -\tfrac{1}{2}\,].$$

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Gaussian Elimination (cont.)



Ejemplo

$$\begin{aligned} 2x_1 - x_2 + x_3 &= 1 \\ 4x_1 + x_2 - x_3 &= 5 \\ x_1 + x_2 + x_3 &= 0. \end{aligned}$$



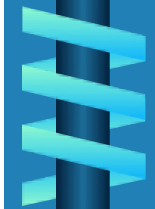
$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 4 & 1 & -1 & 5 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Eliminar x1 de la fila 3: Calculamos el factor $\frac{1}{2}$ y hacemos

$$R_3 \leftarrow R_3 - \frac{1}{2}R_1$$

$$\frac{1}{2}R_1 = [\,1,\ -\tfrac{1}{2}, \tfrac{1}{2} \mid \tfrac{1}{2}\,],$$

$$R_3 - \tfrac{1}{2}R_1 = [\,1-1,\ 1-(-\tfrac{1}{2}),\ 1-\tfrac{1}{2} \mid 0-\tfrac{1}{2}\,] = [\,0, \tfrac{3}{2}, \tfrac{1}{2} \mid\ -\tfrac{1}{2}\,].$$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Gaussian Elimination (cont.)



Ejemplo

$$\begin{aligned} 2x_1 - x_2 + x_3 &= 1 \\ 4x_1 + x_2 - x_3 &= 5 \\ x_1 + x_2 + x_3 &= 0. \end{aligned}$$

$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 4 & 1 & -1 & 5 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

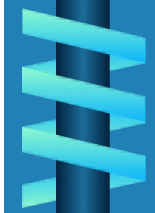
Eliminar x1 de la fila 3: Calculamos el factor $\frac{1}{2}$ y hacemos

$$R_3 \leftarrow R_3 - \frac{1}{2}R_1$$

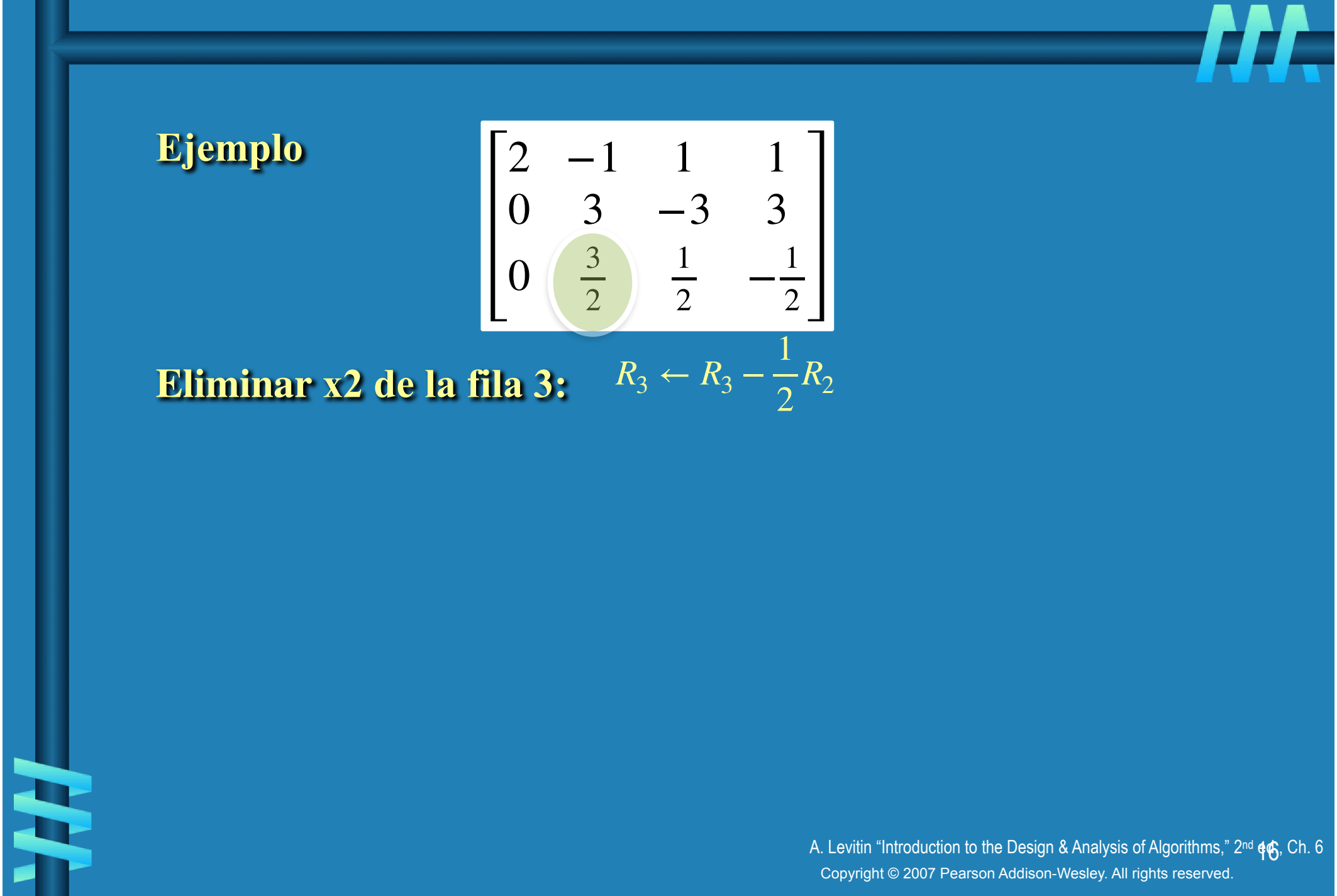
$$\frac{1}{2}R_1 = [\,1,\ -\tfrac{1}{2}, \tfrac{1}{2} \mid \tfrac{1}{2}\,],$$

$$R_3 - \tfrac{1}{2}R_1 = [\,1-1,\ 1-(-\tfrac{1}{2}),\ 1-\tfrac{1}{2} \mid 0-\tfrac{1}{2}\,] = [\,0, \tfrac{3}{2}, \tfrac{1}{2} \mid$$

$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & \frac{3}{2} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.



Ejemplo

$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & \frac{3}{2} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

Eliminar x2 de la fila 3: $R_3 \leftarrow R_3 - \frac{1}{2} R_2$

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

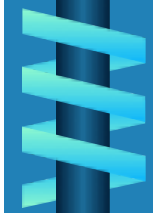
Gaussian Elimination (cont.)



Ejemplo

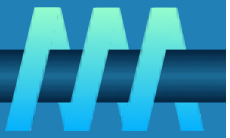
$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & \frac{3}{2} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

Eliminar x2 de la fila 3: $R_3 \leftarrow R_3 - \frac{1}{2} R_2$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Gaussian Elimination (cont.)



Ejemplo

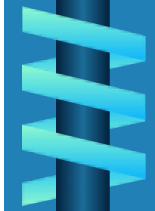
$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & \frac{3}{2} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

Eliminar x2 de la fila 3: $R_3 \leftarrow R_3 - \frac{1}{2} R_2$

$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & 0 & 2 & -2 \end{bmatrix}$$



$$\begin{aligned} 2x_1 - x_2 + x_3 &= 1, \\ 3x_2 - 3x_3 &= 3, \\ 2x_3 &= -2. \end{aligned}$$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Gaussian Elimination (cont.)



Ejemplo

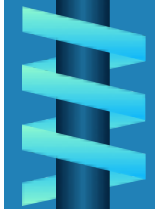
$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & \frac{3}{2} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

Eliminar x2 de la fila 3: $R_3 \leftarrow R_3 - \frac{1}{2} R_2$

Instancia simplificada

$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & 0 & 2 & -2 \end{bmatrix}$$

$$\begin{aligned} 2x_1 - x_2 + x_3 &= 1, \\ 3x_2 - 3x_3 &= 3, \\ 2x_3 &= -2. \end{aligned}$$



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Gaussian Elimination (cont.)

Ejemplo

$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & \frac{3}{2} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

Resolvemos mediante sustitución hacia atras

Eliminar x2 de la fila 3: $R_3 \leftarrow R_3 - \frac{1}{2} R_2$

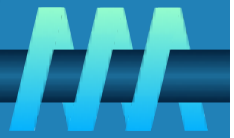
Instancia simplificada

$$\begin{bmatrix} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & 0 & 2 & -2 \end{bmatrix}$$

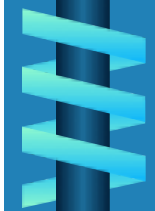
$$\begin{aligned} 2x_1 - x_2 + x_3 &= 1, \\ 3x_2 - 3x_3 &= 3, \\ 2x_3 &= -2. \end{aligned}$$

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Taxonomia de los Algoritmos de Búsqueda



- List searching
  - sequential search
  - binary search
  - interpolation search
- Tree searching
  - binary search tree
  - binary balanced trees: AVL trees, red-black trees
  - multiway balanced trees: 2-3 trees, 2-3-4 trees, B trees
- Hashing

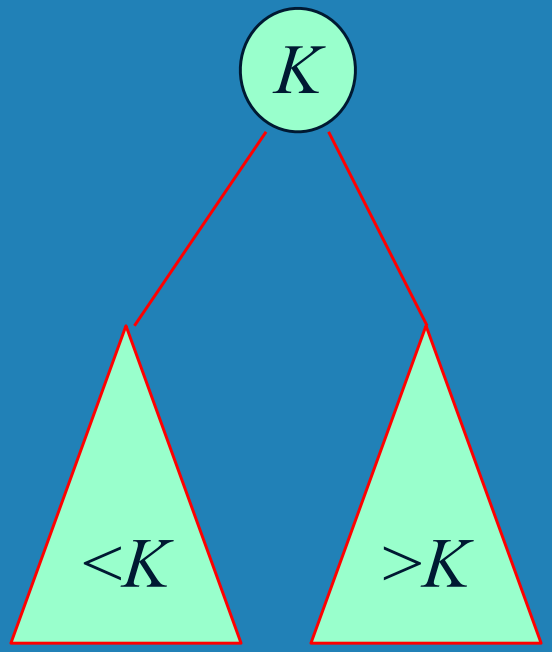


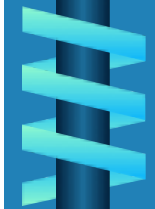
Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Representation Change



¿Qué ganamos transformado de implementación directa de un diccionario mediante, por ejemplo, un arreglo (array) a una implementación basada en Árboles binarios de búsqueda?





Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Instance/Representation Change

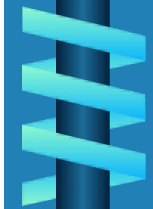


Ganamos en eficiencia temporal en las operaciones de búsqueda, inserción y eliminación, ya que todas tienen un tiempo de ejecución de O(log n) pero **solo en el caso promedio**.

En el peor de los casos, estas operaciones pueden requerir un tiempo de O(n)

Se ha dedicado mucho esfuerzo a intentar encontrar una estructura que conserve las buenas propiedades del árbol binario de búsqueda clásico

pero que evite su degeneración en el peor de los casos.



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

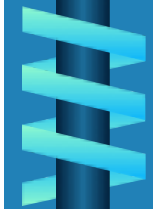
Árboles de Búsqueda Balanceados



La **atractividad de los árboles binarios de búsqueda** se ve afectada por su **ineficiencia en el peor caso (lineal). Dos ideas para superar este problema son:**

- **to rebalance binary search tree when a new insertion/deletion**
  - ***AVL trees***
  - ***red-black trees***

- **to allow more than one key per node of a search tree**
  - ***2-3 trees***
  - ***B-trees***



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Árboles de Búsqueda Balanceados



La **atractividad de los árboles binarios de búsqueda** se ve afectada por su **ineficiencia en el peor caso (lineal). Dos ideas para superar este problema son:**

- **to rebalance binary search tree when a new insertion/deletion**
  - ***AVL trees***
  - ***red-black trees***

Instance simplification

- **to allow more than one key per node of a search tree**
  - ***2-3 trees***
  - ***B-trees***



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Árboles de Búsqueda Balanceados



La **atractividad de los árboles binarios de búsqueda** se ve afectada por su **ineficiencia en el peor caso (lineal). Dos ideas para superar este problema son:**

- **to rebalance binary search tree when a new insertion/deletion**
  - *AVL trees*
  - *red-black trees*

Instance simplification

- **to allow more than one key per node of a search tree**
  - *2-3 trees*
  - *B-trees*

Representation Change

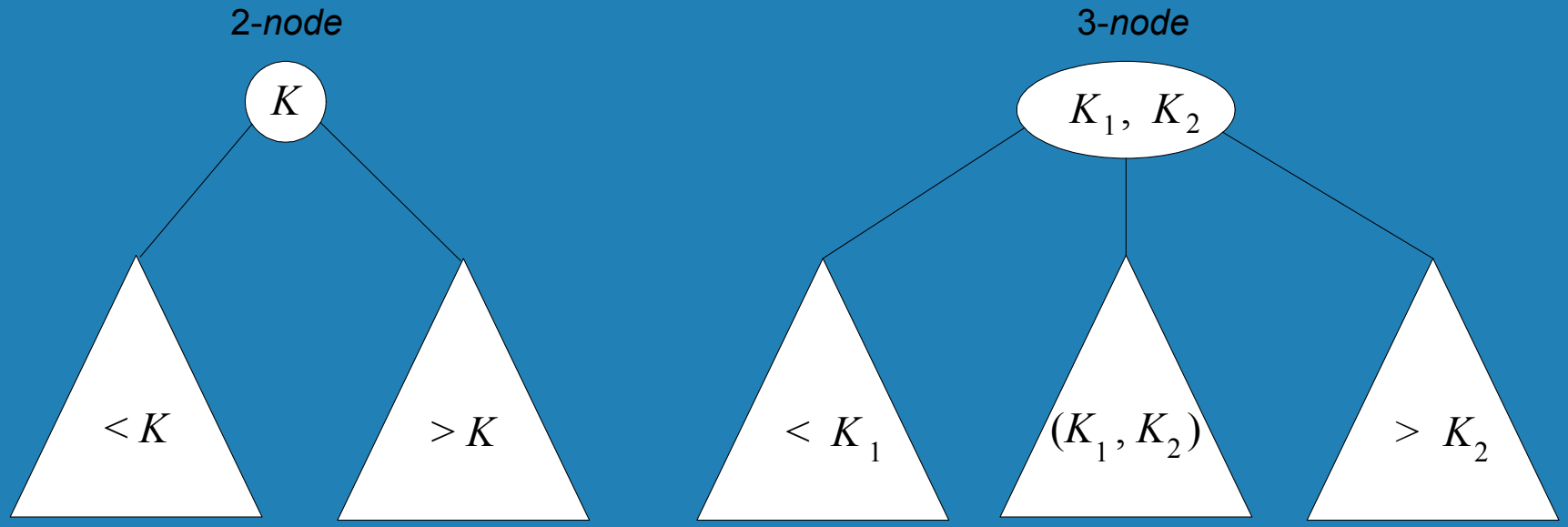


Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

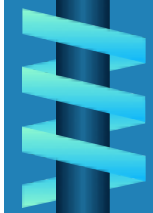
2-3 Tree



Definición:
Un árbol 2–3 es un árbol de búsqueda que puede tener nodos de 2 y nodos de 3, y que está balanceado en altura (todas las hojas se encuentran en el mismo nivel).

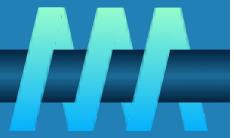


Un árbol 2–3 se construye mediante inserciones sucesivas de claves dadas, insertando siempre la nueva clave en una hoja del árbol.
Si la hoja es un nodo de 3, este se divide en dos, y la clave del medio se promueve al nodo padre.



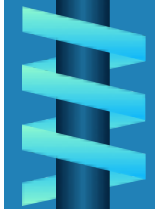
Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Problem Reduction



Esta variación del método **transformar y conquistar (transform-and-conquer)** resuelve un problema **transformándolo en otro problema diferente** para el cual **ya existe un algoritmo disponible**.

Para que esto tenga valor práctico, el tiempo total combinado de la transformación y la resolución del otro problema debe ser menor que el tiempo necesario para resolver el problema original directamente mediante otro método.



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Ejemplos de resolución de Problemas por Reduccción

- Calcular el mínimo común múltiplo usando el máximo común divisor

En lugar de calcular directamente el MCM (LCM) de dos números m y n se transforma el problema en uno de máximo común divisor (GCD), que se puede resolver fácilmente con el algoritmo de Euclides.

$$LCM(m,n) = \frac{m x n}{GCD(m,n)}$$

- Transformar un problema de maximización en uno de minimización y viceversa

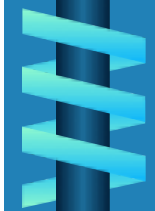
Muchos algoritmos están diseñados para minimizar (por ejemplo, costo, distancia, tiempo).

Pero si tu problema es de maximización (por ejemplo, ganancia, beneficio), podés transformarlo así: Maximizar $f(x)$ $\Leftrightarrow$ Minimizar $-f(x)$

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.



- **Reducción a problemas de grafos, por ejemplo resolver rompecabezas usando grafos de estados, calcular como me conviene cursar según el plan de correlatividades de mi carrera,** encontrar la **ruta más rápida o más barata** entre dos ciudades, etc
- Reducir un problema específico a un matroide
- …



Copyright © 2007 Pearson Addison-Wesley. All rights reserved.

Examples of Solving Problems by Reduction

- **Reducción a problemas de grafos, por ejemplo resolver rompecabezas usando grafos de estados, calcular como me conviene cursar según el plan de correlatividades de mi carrera,** encontrar la **ruta más rápida o más barata** entre dos ciudades, etc
- Reducir un problema específico a un matroide
- …

Copyright © 2007 Pearson Addison-Wesley. All rights reserved.